# Circular Singly Linked List- Application

Write a C program to implement a **Circular Singly Linked List (CSLL)** to manage a Netflix playlist of shows. Each node should store details such as Show ID, Show Name, and Rating. The program should allow the user to create a circular linked list of *n* shows and perform various operations such as inserting a new show at the beginning and at the end of the playlist, as well as inserting a show after the first show with an even Show ID. Additionally, the program should display all shows in the circular playlist, display only shows with even Show IDs, find and display the show with the highest rating, and search for a show using its Show ID. The program must be menu-driven and should use dynamic memory allocation while maintaining the circular structure of the linked list.

```
#include<stdio.h>
#include<stdlib.h>

struct show
{
    int id;
    float rating;
    char name[20];
    struct show *next;
};

/* Create CSLL */
struct show *create(int n)
{
    struct show *head=NULL,*temp=NULL,*newnode;
    int i;

    for(i=1;i<=n;i++)
    {
        newnode=(struct show*)malloc(sizeof(struct
show));

        printf("\nEnter Show ID: ");
        scanf("%d",&newnode->id);

        printf("Enter Show Name: ");
        scanf("%s",newnode->name);

        printf("Enter Rating: ");
        scanf("%f",&newnode->rating);

        if(head==NULL)
        {
            head=newnode;
            newnode->next=head;
            temp=newnode;
        }
        else
        {
            temp->next=newnode;
            newnode->next=head;
            temp=newnode;
        }
    }

    return head;
}
```

```c
/* Display CSLL */
void display(struct show *head)
{
    struct show *temp;

    if(head==NULL)
    {
        printf("Playlist empty\n");
        return;
    }

    temp=head;

    printf("\nNetflix Playlist:\n");

    do
    {
        printf("ID:%d Name:%s Rating:%.1f\n",
        temp->id,temp->name,temp->rating);

        temp=temp->next;
    }while(temp!=head);
}

/* Insert at Beginning */
struct show* insert_begin(struct show *head)
{
    struct show *newnode,*temp;

    newnode=(struct show*)malloc(sizeof(struct show));

    printf("Enter Show ID: ");
    scanf("%d",&newnode->id);

    printf("Enter Show Name: ");
    scanf("%s",newnode->name);

    printf("Enter Rating: ");
    scanf("%f",&newnode->rating);

    if(head==NULL)
    {
        head=newnode;
        newnode->next=head;
        return head;
    }
```

```
    temp=head;

    while(temp->next!=head)
        temp=temp->next;

    newnode->next=head;
    temp->next=newnode;
    head=newnode;

    return head;
}

/* Insert at End */
struct show* insert_end(struct show *head)
{
    struct show *newnode,*temp=head;

    newnode=(struct show*)malloc(sizeof(struct show));

    printf("Enter Show ID: ");
    scanf("%d",&newnode->id);

    printf("Enter Show Name: ");
    scanf("%s",newnode->name);

    printf("Enter Rating: ");
    scanf("%f",&newnode->rating);

    if(head==NULL)
    {
        head=newnode;
        newnode->next=head;
        return head;
    }

    while(temp->next!=head)
        temp=temp->next;

    temp->next=newnode;
    newnode->next=head;

    return head;
}
```

```
/* Insert after even ID */
struct show* insert_after_even(struct show *head)
{
    struct show *temp=head,*newnode;

    if(head==NULL)
    {
        printf("List empty\n");
        return head;
    }

    do
    {
        if(temp->id%2==0)
        {
            newnode=(struct show*)malloc(sizeof(struct
show));

            printf("Enter Show ID: ");
            scanf("%d",&newnode->id);

            printf("Enter Show Name: ");
            scanf("%s",newnode->name);

            printf("Enter Rating: ");
            scanf("%f",&newnode->rating);

            newnode->next=temp->next;
            temp->next=newnode;

            return head;
        }

        temp=temp->next;

    }while(temp!=head);

    printf("No even ID found\n");
    return head;
}

/* Display Even ID Shows */
void even_id(struct show *head)
{
    struct show *temp=head;

    if(head==NULL) return;
```

```
    printf("\nEven ID Shows:\n");

    do
    {
        if(temp->id%2==0)
        printf("ID:%d Name:%s Rating:%.1f\n",
        temp->id,temp->name,temp->rating);

        temp=temp->next;

    }while(temp!=head);
}

/* Max Rating */
void max_rating(struct show *head)
{
    struct show *temp=head,*max=head;

    if(head==NULL)
    {
        printf("List Empty\n");
        return;
    }

    do
    {
        if(temp->rating > max->rating)
            max=temp;

        temp=temp->next;

    }while(temp!=head);

    printf("\nTop Rated Show:\n");
    printf("ID:%d Name:%s Rating:%.1f\n",
    max->id,max->name,max->rating);
}

/* Search by ID */
void search(struct show *head)
{
    int id;
    struct show *temp=head;

    if(head==NULL)
    {
```

```
        printf("List empty\n");
        return;
    }

    printf("Enter Show ID to search: ");
    scanf("%d",&id);

    do
    {
        if(temp->id==id)
        {
            printf("\nShow Found:\n");
            printf("ID:%d Name:%s Rating:%.1f\n",
            temp->id,temp->name,temp->rating);
            return;
        }

        temp=temp->next;

    }while(temp!=head);

    printf("Show not found\n");
}

/* Main */
int main()
{
    struct show *head=NULL;
    int n,ch;

    printf("Enter number of shows: ");
    scanf("%d",&n);

    head=create(n);

    while(1)
    {
        printf("\n1.Insert Begin");
        printf("\n2.Insert End");
        printf("\n3.Insert After Even ID");
        printf("\n4.Display");
        printf("\n5.Display Even ID Shows");
        printf("\n6.Max Rating");
        printf("\n7.Search Show");
        printf("\n8.Exit");

        printf("\nEnter choice: ");
```

```
        scanf("%d",&ch);

        switch(ch)
        {
            case 1: head=insert_begin(head); break;
            case 2: head=insert_end(head); break;
            case 3: head=insert_after_even(head); break;
            case 4: display(head); break;
            case 5: even_id(head); break;
            case 6: max_rating(head); break;
            case 7: search(head); break;
            case 8: exit(0);
            default: printf("Invalid choice");
        }
    }
}
```